

DSA patterns cheat sheet

For students preparing for coding rounds: a Big-O reference, then 14 common patterns, each with its signals, a JavaScript template, complexity and classic problems.

Most coding-round questions are versions of a few patterns. Spot the pattern from the **signals** in the problem, write the **template** from memory, then adapt it. All templates are plain JavaScript and run in Node.js as written. Problem names are the common names used on practice sites.

Big-O reference

Common operations

Structure	Operation	Time
Array	Read / write by index	$O(1)$
	Search for a value	$O(n)$; $O(\log n)$ with binary search if sorted
	Add / remove at end (<code>push</code> , <code>pop</code>)	$O(1)$ amortised
	Add / remove at start or middle (<code>unshift</code> , <code>shift</code> , <code>splice</code>)	$O(n)$
Hash map / set (<code>Map</code> , <code>Set</code>)	Insert, lookup, delete	$O(1)$ average; $O(n)$ worst case (many collisions)
Binary heap	Peek min / max	$O(1)$
	Push, pop; build from n items	$O(\log n)$; $O(n)$
Binary search tree	Search, insert, delete	$O(\log n)$ if balanced (AVL, red-black); $O(n)$ worst if unbalanced
Linked list	Insert / delete at a known node; access k -th or search	$O(1)$; $O(n)$

Sorting

Algorithm	Best	Average	Worst	Extra space	Stable?
Bubble / insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$ stack	No
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting (values 0..k)	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	Yes

Tip: In JavaScript, `[10, 9, 1].sort()` gives `[1, 10, 9]` because the default sort compares values as strings. For numbers always write `arr.sort((a, b) => a - b)`. The built-in sort is stable (required since ES2019) and $O(n \log n)$ in modern engines.

What the input size allows (rough guide)

A common rule of thumb is about 10^8 simple operations per second, so the limit on n hints at the expected complexity:

n up to about	10	20	500	5,000	10^5 to 10^6	10^9 or more
Target	$O(n!)$	$O(2^n)$	$O(n^3)$	$O(n^2)$	$O(n \log n)$ or $O(n)$	$O(\log n)$ or $O(1)$

1. Two pointers

Signals: sorted array; find a pair or triplet with a target sum; palindrome check; reverse or remove duplicates in place. **Idea:** one pointer at each end (or a slow and a fast one); each move rules out many candidates at once.

```
// Sorted array: indices of two numbers that add up to target
function pairWithSum(arr, target) {
  let lo = 0, hi = arr.length - 1;
  while (lo < hi) {
    const sum = arr[lo] + arr[hi];
    if (sum === target) return [lo, hi];
    if (sum < target) lo++; // need a bigger sum
    else hi--; // need a smaller sum
  }
  return [-1, -1];
}
```

Complexity: $O(n)$ time, $O(1)$ space (plus $O(n \log n)$ if you sort first). **Practise:** Two Sum II, 3Sum, Container With Most Water, Valid Palindrome.

2. Sliding window

Signals: "contiguous subarray" or "substring"; longest, shortest or maximum; "at most k distinct"; window of size k. **Idea:** grow the window on the right; when it breaks a rule, shrink it from the left. Each element enters and leaves once. For a *fixed* window of size k, add `arr[i]` and subtract `arr[i - k]` at each step.

```
// Variable window: longest substring without repeating characters
function longestUniqueSubstring(s) {
  const lastSeen = new Map();
  let left = 0, best = 0;
  for (let right = 0; right < s.length; right++) {
    const ch = s[right];
    if (lastSeen.has(ch) && lastSeen.get(ch) >= left) {
      left = lastSeen.get(ch) + 1; // jump past the repeat
    }
    lastSeen.set(ch, right);
    best = Math.max(best, right - left + 1);
  }
  return best;
}
```

Complexity: $O(n)$ time; $O(\text{size of character set})$ space. **Practise:** Longest Substring Without Repeating Characters, Minimum Window Substring, Longest Repeating Character Replacement.

Tip: Sliding window needs "shrinking can only make it more valid". Sum problems with negative numbers break this; use prefix sums instead.

3. Prefix sums

Signals: many range-sum queries; "number of subarrays with sum k"; negatives present. **Idea:** `prefix[i]` is the sum of the first i elements, so the sum of `arr[l..r]` is `prefix[r + 1] - prefix[l]`. A subarray ending here sums to k when an earlier prefix equals `sum - k`.

```
function buildPrefix(arr) {
  const prefix = new Array(arr.length + 1).fill(0);
  for (let i = 0; i < arr.length; i++) prefix[i + 1] = prefix[i] + arr[i];
  return prefix;
}
const rangeSum = (prefix, l, r) => prefix[r + 1] - prefix[l]; // inclusive l..r
```

```
// Count subarrays whose sum is exactly k (works with negatives)
function subarraySumEqualsK(nums, k) {
  const count = new Map([[0, 1]]); // the empty prefix has sum 0
  let sum = 0, result = 0;
  for (const x of nums) {
    sum += x;
    result += count.get(sum - k) || 0;
    count.set(sum, (count.get(sum) || 0) + 1);
  }
  return result;
}
```

Complexity: build $O(n)$, each query $O(1)$; counting $O(n)$ time and space. **Practise:** Range Sum Query (Immutable), Subarray Sum Equals K, Product of Array Except Self.

4. Hashing

Signals: unsorted data needing fast lookup; "seen before?"; frequencies; duplicates; grouping by a key. **Idea:** trade memory for time: a `Map` or `Set` turns an $O(n)$ search into an $O(1)$ average lookup.

```
// Two Sum (unsorted): indices of two numbers that add to target
function twoSum(nums, target) {
  const indexOf = new Map(); // value -> index
  for (let i = 0; i < nums.length; i++) {
    const need = target - nums[i];
    if (indexOf.has(need)) return [indexOf.get(need), i];
    indexOf.set(nums[i], i);
  }
  return [];
}
```

For grouping (Group Anagrams), use a key such as `w.split('').sort().join('')` and a `Map` from key to a list. Prefer `Map` over `{}`: object keys are always strings, so `1` and `"1"` collide.

Complexity: Two Sum $O(n)$ time and space. **Practise:** Two Sum, Group Anagrams, Contains Duplicate, Longest Consecutive Sequence.

5. Binary search (and binary search on the answer)

Signals: sorted data; "first / last position"; $O(\log n)$ required; or "minimum speed / capacity / days such that..." where once an answer works, every bigger one works too. **Idea:** keep a range that surely holds the answer and halve it. "On the answer" means searching over answer values with a yes/no check.

```
// First index i with arr[i] >= target (arr.length if none)
function lowerBound(arr, target) {
  let lo = 0, hi = arr.length;
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (arr[mid] < target) lo = mid + 1;
    else hi = mid;
  }
  return lo;
}
```

```
// Koko Eating Bananas: smallest speed that finishes all piles within h hours
function minEatingSpeed(piles, h) {
  const canFinish = (speed) => {
    let hours = 0;
    for (const p of piles) hours += Math.ceil(p / speed);
    return hours <= h;
  };
  let lo = 1, hi = Math.max(...piles);
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (canFinish(mid)) hi = mid; // mid works, try smaller
    else lo = mid + 1; // too slow
  }
  return lo;
}
```

Complexity: $O(\log n)$ time, $O(1)$ space; on the answer $O(n \log M)$, where M is the answer range (largest pile).

Practise: Search in Rotated Sorted Array, Find First and Last Position of Element, Koko Eating Bananas, Capacity To Ship Packages Within D Days, Allocate Books.

Tip: Stick to one style. The `lo < hi` loop with `hi = mid / lo = mid + 1` always ends with `lo === hi`. Test on arrays of size 0, 1 and 2.

6. Stack and monotonic stack

Signals: matching brackets; expression evaluation; "next greater / smaller element"; "days until a warmer temperature"; histogram areas. **Idea:** the stack holds items still waiting for an answer. A monotonic stack keeps them sorted; a new item that breaks the order answers the items it pops. For brackets: push openers, pop and compare on closers, and the stack must end empty.

```
// Next greater element to the right (-1 if none)
function nextGreater(nums) {
  const result = new Array(nums.length).fill(-1);
  const stack = []; // indices; their values are decreasing
  for (let i = 0; i < nums.length; i++) {
    while (stack.length && nums[stack[stack.length - 1]] < nums[i]) {
      result[stack.pop()] = nums[i];
    }
    stack.push(i);
  }
  return result;
}
```

Complexity: $O(n)$ time (each index is pushed and popped at most once), $O(n)$ space. **Practise:** Valid Parentheses, Daily Temperatures, Largest Rectangle in Histogram, Min Stack.

7. Linked list: fast and slow pointers

Signals: cycle; middle node; k-th from the end; palindrome list; "O(1) extra space". **Idea:** fast moves two steps per slow step. With a cycle they meet; without one, fast reaches the end when slow is at the middle (the second middle for even length).

```
class ListNode {
  constructor(val, next = null) { this.val = val; this.next = next; }
}
```

```
function hasCycle(head) {
  let slow = head, fast = head;
  while (fast && fast.next) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow === fast) return true;
  }
  return false;
}
```

```
function reverseList(head) {
  let prev = null, curr = head;
  while (curr) {
    const next = curr.next;
    curr.next = prev;
    prev = curr;
    curr = next;
  }
  return prev;
}
```

Complexity: $O(n)$ time, $O(1)$ space. **Practise:** Linked List Cycle II, Middle of the Linked List, Reverse Linked List, Palindrome Linked List, Remove Nth Node From End.

8. Recursion and backtracking

Signals: "generate all" subsets, permutations or combinations; N-Queens, Sudoku; small n (about 20 or less).

Idea: build the answer one choice at a time: *choose, explore, un-choose*. Prune branches that cannot succeed.

```
function subsets(nums) {
  const result = [], path = [];
  function backtrack(start) {
    result.push([...path]); // save a COPY of the path
    for (let i = start; i < nums.length; i++) {
      path.push(nums[i]); // choose
      backtrack(i + 1); // explore
      path.pop(); // un-choose
    }
  }
  backtrack(0);
  return result;
}
```

For permutations, loop from 0 each time, skip elements already used (a `used` array), and save only when the path has n elements.

Complexity: subsets $O(n \cdot 2^n)$, permutations $O(n \cdot n!)$; $O(n)$ recursion depth. **Practise:** Subsets, Permutations, Combination Sum, N-Queens, Word Search.

9. BFS and DFS on grids and graphs

Signals: grid of cells; islands or regions; connected components; "minimum number of steps"; flood fill. **Idea:**

DFS goes deep and is simplest for "visit everything connected". BFS goes level by level, so it finds the fewest steps in an *unweighted* grid or graph.

```
// DFS: count islands of '1' in a grid of '0'/'1' strings
function numIslands(grid) {
  const rows = grid.length, cols = grid[0].length;
  let count = 0;
  function dfs(r, c) {
    if (r < 0 || c < 0 || r >= rows || c >= cols || grid[r][c] !== '1') return;
    grid[r][c] = '0'; // mark visited
    dfs(r + 1, c); dfs(r - 1, c); dfs(r, c + 1); dfs(r, c - 1);
  }
  for (let r = 0; r < rows; r++)
    for (let c = 0; c < cols; c++)
      if (grid[r][c] === '1') { count++; dfs(r, c); }
  return count;
}
```

```
// BFS: fewest steps from start to end; grid 0 = open, 1 = wall
function shortestPath(grid, start, end) {
  const rows = grid.length, cols = grid[0].length;
  const dirs = [[1, 0], [-1, 0], [0, 1], [0, -1]];
  const dist = Array.from({ length: rows }, () => new Array(cols).fill(-1));
  const queue = [start];
  dist[start[0]][start[1]] = 0;
  let head = 0; // index pointer: shift() is O(n)
  while (head < queue.length) {
    const [r, c] = queue[head++];
    if (r === end[0] && c === end[1]) return dist[r][c];
    for (const [dr, dc] of dirs) {
      const nr = r + dr, nc = c + dc;
      if (nr >= 0 && nc >= 0 && nr < rows && nc < cols &&
        grid[nr][nc] === 0 && dist[nr][nc] === -1) {
        dist[nr][nc] = dist[r][c] + 1; // mark when you ADD to the queue
        queue.push([nr, nc]);
      }
    }
  }
  return -1; // unreachable
}
```

Complexity: grid $O(R \times C)$ time and space; graph (adjacency list) $O(V + E)$. **Practise:** Number of Islands, Rotting Oranges (multi-source BFS), Shortest Path in Binary Matrix, Clone Graph, Number of Provinces. On very large grids, recursive DFS can overflow the call stack; use BFS or an explicit stack.

10. Topological sort

Signals: prerequisites; "course schedule"; build order; cycle in a *directed* graph. **Idea (Kahn's algorithm):** start with nodes that have no incoming edges; remove them one by one, lowering their neighbours' in-degree. Nodes never reached mean a cycle.

```
// n nodes 0..n-1; edge [u, v] means u must come before v
function topoSort(n, edges) {
  const adj = Array.from({ length: n }, () => []);
  const indegree = new Array(n).fill(0);
  for (const [u, v] of edges) { adj[u].push(v); indegree[v]++; }
  const queue = [];
  for (let i = 0; i < n; i++) if (indegree[i] === 0) queue.push(i);
  const order = [];
  let head = 0;
  while (head < queue.length) {
    const u = queue[head++];
    order.push(u);
    for (const v of adj[u]) if (--indegree[v] === 0) queue.push(v);
  }
  return order.length === n ? order : []; // [] means there is a cycle
}
```

Complexity: $O(V + E)$ time and space. **Practise:** Course Schedule I and II, Alien Dictionary, Parallel Courses.

11. Heaps and top-K

Signals: "k-th largest / smallest"; "top k frequent"; merge k sorted lists; running median. **Idea:** for the k largest, keep a *min*-heap of size k; its top is the smallest of your top k, cheap to drop.

```
function kthLargest(nums, k) {
  const heap = new MinHeap();          // holds the k largest so far
  for (const x of nums) {
    heap.push(x);
    if (heap.size() > k) heap.pop();    // drop the smallest
  }
  return heap.peek();
}
```

JavaScript has no built-in heap, so be ready to write one. This min-heap compares numbers; for objects, change the comparisons.

```
class MinHeap {
  constructor() { this.a = []; }
  size() { return this.a.length; }
  peek() { return this.a[0]; }
  push(x) {
    const a = this.a;
    a.push(x);
    let i = a.length - 1;
    while (i > 0 && a[(i - 1) >> 1] > a[i]) { // bubble up
      const p = (i - 1) >> 1;
      [a[p], a[i]] = [a[i], a[p]];
      i = p;
    }
  }
  pop() {
    const a = this.a, top = a[0], last = a.pop();
    if (a.length === 0) return top;
    a[0] = last;
    let i = 0;
    while (true) { // sink down
      const l = 2 * i + 1, r = l + 1;
      let small = i;
      if (l < a.length && a[l] < a[small]) small = l;
      if (r < a.length && a[r] < a[small]) small = r;
      if (small === i) return top;
      [a[small], a[i]] = [a[i], a[small]];
      i = small;
    }
  }
}
```

Complexity: push / pop $O(\log n)$; top-K $O(n \log k)$ time, $O(k)$ space. (Sorting everything, $O(n \log n)$, is often fine too; say so.) **Practise:** Kth Largest Element in an Array, Top K Frequent Elements, Merge k Sorted Lists, Find Median from Data Stream.

12. Intervals

Signals: [start, end] pairs; meetings, bookings; "merge", "insert", "overlap", "minimum rooms". **Idea:** sort first. Sort by start to merge. To keep the most non-overlapping intervals, sort by end and greedily keep each one that starts at or after the last kept end.

```
function mergeIntervals(intervals) {
  if (intervals.length === 0) return [];
  const sorted = [...intervals].sort((a, b) => a[0] - b[0]);
  const merged = [sorted[0].slice()];
  for (let i = 1; i < sorted.length; i++) {
    const last = merged[merged.length - 1];
    const [start, end] = sorted[i];
    if (start <= last[1]) last[1] = Math.max(last[1], end); // overlap
    else merged.push([start, end]);
  }
  return merged;
}
```

Complexity: $O(n \log n)$ time (the sort), $O(n)$ space. **Practise:** Merge Intervals, Insert Interval, Non-overlapping Intervals, Meeting Rooms II (min-heap of end times). Always ask whether touching intervals like [1, 4] and [4, 5] overlap; the code above says yes.

13. Dynamic programming

Signals: "number of ways"; "minimum cost" or "maximum profit"; choices at each step; plain recursion repeats the same sub-problems. **Recipe:** (1) define the *state* (what `dp[i]` means, in one sentence); (2) the *transition* from smaller states; (3) the *base case*; (4) a fill *order* so needed states are ready. Recursion plus a cache is top-down (memoization); loops filling a table are bottom-up (tabulation), which avoids stack overflow.

```
// 1D: House Robber, no two adjacent houses
// dp[i] = max(dp[i-1], dp[i-2] + nums[i]); keep only the last two values
function rob(nums) {
  let prev2 = 0, prev1 = 0;
  for (const x of nums) {
    const curr = Math.max(prev1, prev2 + x);
    prev2 = prev1;
    prev1 = curr;
  }
  return prev1;
}
```

```
// 2D: Longest Common Subsequence
// dp[i][j] = LCS of a's first i chars and b's first j chars
function lcs(a, b) {
  const m = a.length, n = b.length;
  const dp = Array.from({ length: m + 1 }, () => new Array(n + 1).fill(0));
  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
      if (a[i - 1] === b[j - 1]) dp[i][j] = dp[i - 1][j - 1] + 1;
      else dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
    }
  }
  return dp[m][n];
}
```

```
// 0/1 knapsack: each item at most once; dp[w] = best value within weight w
function knapsack(weights, values, capacity) {
  const dp = new Array(capacity + 1).fill(0);
  for (let i = 0; i < weights.length; i++) {
    for (let w = capacity; w >= weights[i]; w--) { // DOWN: item i used once
      dp[w] = Math.max(dp[w], dp[w - weights[i]] + values[i]);
    }
  }
  return dp[capacity];
}
// knapsack([1, 3, 4, 5], [1, 4, 5, 7], 7) returns 9 (weights 3 + 4)
```

Complexity: House Robber $O(n)$ time, $O(1)$ space; LCS $O(m \times n)$ time and space; knapsack $O(\text{items} \times \text{capacity})$ time, $O(\text{capacity})$ space. Coin Change (fewest coins) is $\text{dp}[a] = \min(\text{dp}[a - c] + 1)$ over coins $c \leq a$, with $\text{dp}[0] = 0$: $O(\text{amount} \times \text{coins})$.

Tip: Looping w *upwards* in the 1D knapsack lets an item be reused: that is the unbounded knapsack (like Coin Change). Know which one the problem wants.

Practise: Climbing Stairs, House Robber, Coin Change, Longest Increasing Subsequence, Edit Distance, Unique Paths, Partition Equal Subset Sum.

14. Greedy

Signals: "minimum number of jumps / platforms"; scheduling; taking the best-looking option now never hurts later. **Idea:** make the locally best choice and never go back. It is correct only if you can justify it. Check for a counter-example first: with coins $[1, 3, 4]$ and amount 6, greedy takes $4 + 1 + 1$ (3 coins) but $3 + 3$ (2 coins) is better, so that problem needs DP.

```
// Jump Game: can you reach the last index?
function canJump(nums) {
  let farthest = 0;
  for (let i = 0; i < nums.length; i++) {
    if (i > farthest) return false; // stuck before reaching i
    farthest = Math.max(farthest, i + nums[i]);
  }
  return true;
}
```

```
// Kadane's algorithm: maximum subarray sum (non-empty array)
function maxSubarray(nums) {
  let best = nums[0], curr = 0;
  for (const x of nums) {
    curr = Math.max(x, curr + x); // extend, or start fresh at x
    best = Math.max(best, curr);
  }
  return best;
}
```

Complexity: $O(n)$ time, $O(1)$ space; greedy that sorts first is $O(n \log n)$. **Practise:** Jump Game, Best Time to Buy and Sell Stock, Maximum Subarray, Gas Station, Minimum Platforms, Activity Selection.

Last-week checklist

- Write all 14 templates from memory once. Rewrite any you got wrong the next day.
- Write `lowerBound`, grid BFS and `MinHeap` twice: they are the easiest to get subtly wrong.
- Solve one classic problem per pattern, timed (about 25 minutes each).
- Revise the Big-O tables and the input-size guide; state complexity for every solution.
- JavaScript traps: numeric `sort` needs `(a, b) => a - b`; `shift()` is $O(n)$; save copies (`[...path]`) in backtracking; use `Map` for counting.
- Keep an edge-case list: empty input, one element, duplicates, negatives, very large values, no valid answer.
- In the interview: clarify the input, give the brute force, name the pattern and why it fits, code while talking, dry run one edge case, then state complexity unprompted.

PrepSuccess prep guide · Practise these on PrepSuccess: Skill checks, Learn tasks and Interview prep.